补：运算符重载的参数形式

```
double  d1=10;
complex1 A1（30，-40）,C1; //运算符重载为成员函数
complex2 A2 （30，-40）,C2; //运算符重载为友元函数
```

计算：

C1=A1+d1;

C1=d1+A1;

```
complex1 operator  +(double);
complex1 operator  +(complex);
complex1 operator  +(complex &);
complex1 operator  +(const complex &);
```

需要有缺省构造参数，进行类型转换

成员函数默认左操作数是类对象

当引用作为形参时，实参不可能进行类型转换，也不能使用不可寻址的值。

const引用不同，它是只读的，实参也可以使用不可寻址的值，并能进行类型转换

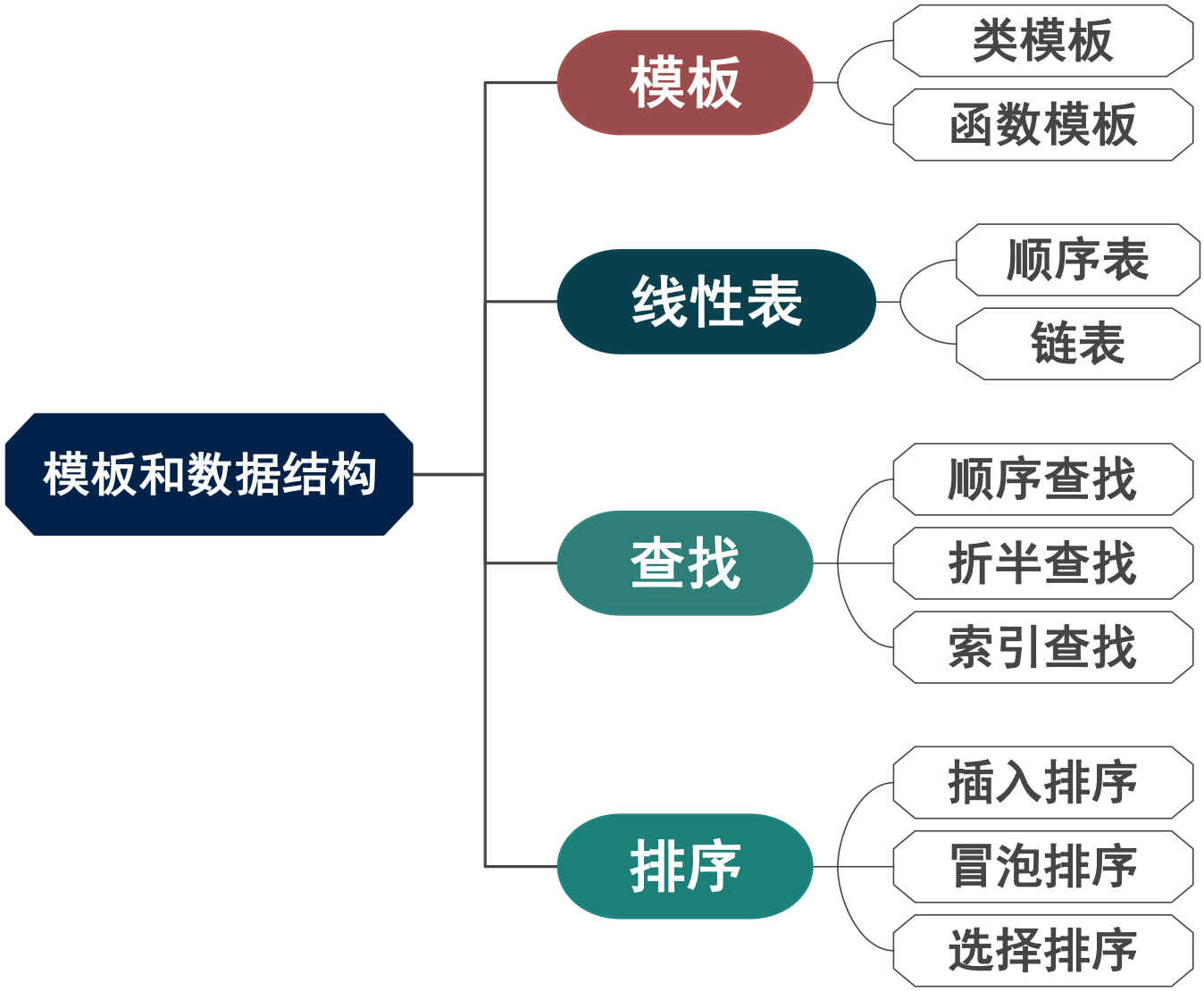
C2=A2+d1;

```
friend complex2 operator+(complex2,double);
friend complex2 operator+(const complex2 &,const complex2 &);
```

C2=d1+A2;

```
friend complex2 operator+(double, complex2);
```

第二讲 模板与数据结构


模板和数据结构
模板
类模板
函数模板
线性表
顺序表
链表
查找
顺序查找
折半查找
索引查找
排序
插入排序
冒泡排序
选择排序

1、模板概念的引入

模板的引入：

增强代码的通用性，使之不受数据类型的限制。

模板的实现：

将数据类型作为参数。

模板(template)
- 函数模板(function ...)
- 类模板(class ...)

2、函数模板

功能：创建一个通用功能的函数，支持多种不同类型的形参，简化重载函数的函数体设计。

例如下三组数据，求每组数据的最大值

```
int      a1，b2，c2
float    a2，b2，c2
double a3，b3，c3
```

函数重载

```
int max(int,int,int);
float max(float,float,float);
double max(double,double,double);
```

```
template<typename T>
T max(T a,T b,T c)
{
        if(b>a) a=b;
        if(c>a) a=c;
        return a;
}
```

功能相同，参数类型不同，三个函数可否用一个通用格式

2、函数模板

函数模板定义：

```
template<模板参数表> 返回类型 函数名(形式参数表)
    {                    …;   }
```

- 函数模板不是类型,也不是函数。
- 编译器遇到模板定义时， 确定模板的实参后，生产实际函数代码，成为模板函数。
- 函数模板根据一组实际类型或者值构造模板函数，其过程通常是隐式的。

2、函数模板

```
int main( )
{
    int   i1=185,  i2=-76,  i3=567,  i;
    double  d1=56.8, d2=90.2,  d3=-314.7,  d;
    long   g1=6785,  g2=-91245,  g3=6734,  g;
    i=max<int>(i1,i2,i3);
    d=max<double>(d1,d2,d3);
    g=max<long>(g1,g2,g3);
    cout<<“i_max=“<<i<<endl;
    cout<<“f_max=“<<d<<endl;
    cout<<“g_max=“<<g<<endl;
    return 0;
}
```

调用模板函数，T被int取代

调用模板函数T被double取代

调用模板函数T被long取代

2、函数模板

例2. 通过函数模板来实现求数的绝对值。

```
#include <iostream>
using namespace std;
template<typename T>T myabs(T x)
    {   return x<0?-x:x;   }
int main()
{
   int n=-5;
   double d=-5.5;
   cout<<myabs(n)<<endl;
   float X=3.0f;
   cout<<myabs(d)<<endl;
  cout<<myabs(X)<<endl;
  return 0;
}
```

```
int myabs(int x)
{   return x<0?-x:x;   }
```

```
double myabs(double x)
{   return x<0?-x:x;   }
```

```
float myabs(float x)
{   return x<0?-x:x;   }
```

2、函数模板

例3. 求数组元素中最大值，通过函数模板来实现。

```
#include <iostream>
#include<string>
using namespace std;
template<typename T>T max(const T *r_array,int size)
{
    T max_val=r_array[0];
        for (int i=1;i<size; ++i)
          if(r_array[i]>max_val)
                max_val=r_array[i];
        return  max_val;
}
```

2、函数模板

例3. 求数组元素中最大值，通过函数模板来实现。

```
int main()
{       int      ia[5]={10,7,14,3,25};
        double da[6]={10.2,7.1,14.5,3.2,25.6,16.8};
        string   sa[5]={"上海","北京","沈阳","广州","武汉"};
        int  i=max(ia,5);
        cout <<"整数最大值为： "<<i<<endl;
        double  d=max(da,6);
        cout <<"实数最大值为： "<<d<<endl;
        string  s=max(sa,5);
        cout <<"字典排序最大为： "<<s<<endl;
        return 0;
}
```

2、函数模板

```
template<typename T>T max(T a,T b){ return a>b?a:b；}
```

```
i=max(35,120);
d=max(172.5, 193.4);
g=max(‘d’, ‘A’ );
```



```
h=max(7.5, 5);
```

这样用可以吗？

error。。。

匹配模板函数时，不进行类型自动转换

2、函数模板

函数模板中可以不止一个类型参。

```
template<typename T1, typename T2>
T1 max(T1 a,T2 b)
{
        if(b>a) return b;
          else return a;
}
```

```
cout<<max(7.5, 5)<<endl;
```

➡ 7.5

```
cout<<max(5, 7.5)<<endl;
```

➡ 7

2、函数模板

在有多个函数和函数模板名字相同情况下，编译器

1）先找参数完全匹配的普通函数；

2）再找参数完全匹配的模板函数；

3）再找实参数经过自动类型转换后能够匹配的普遍函数；

4）上面都找不到，则报错；

匹配模板函数时，不进行类型自动转换

演示示例

2、函数模板

函数模板注意事项

- 用函数模板比函数重载更方便，程序更简洁。
- 但它只适用于函数的参数个数相同而类型不同，且函数体相同的情况，若参数的个数不同，则不能用函数模板。

2、函数模板

例：矩阵运算：矩阵转置函数模板

```
template <typename T1, typename T2>
void inverse(T1 *mat1,T2 *mat2,int a,int b)
{
   int i,j;
    for (i=0;i<b;i++)
      for (j=0;j<a;j++)
        mat2[j][i]=mat1[i][j];
 }
```

2、函数模板

例：矩阵运算：矩阵相乘函数模板

```
template <typename T1, typename T2 >
void multi(T1*mat1,T2 *mat2,T2 *result,int a,int b,int c)
{
        int i,j,k;
         for (i=0;i<a;i++)
            for (j=0;j<c;j++)
           {
                result[i][j] = 0;
                for (k=0;k<b;k++)
                   result[i][j]+=mat1[i][k]*mat2[k][j];
            }
}
```

3、类模板

若有两个或多个类，其功能相同，仅数据类型不同

```
class Compare_int    //对两个整数作比较
{
public:
    Compare_int(int a,int b)
    {   x=a;     y=b;    }
    int max( )
    {     return   (x>y)?x:y;   }
    int min( )
    {    return   (x<y)?x:y;   }
private:
    int x,y;
};
```

若要比较两个浮点数大小呢？

3、类模板

用类模板实现：

虚拟类型numtype

```
template<typename numtype>
class Compare
{
public:
    Compare(numtype a,numtype b)
      {  x=a;  y=b;  }
    numtype max( )
      {  return (x>y) ? x : y;    }
    numtype min( )
      {  return (x<y) ? x : y;    }
 private:
    numtype x,y;
};
```

类模板名为Compare

3、 类模板

类模板定义：

```
template< 模板参数表> class  类模板名
{
        ..... //类体
};
```

分号不要漏掉

在类模板外定义成员函数，应采用以下形式：

```
template<模板参数表>
返回类型 类模板名<模板参数表>::成员函数名 (形参表)
{
        ..... //成员函数体
}
```

使用模板建立对象：

```
类模板名<模板参数表> 对象名1，...，对象名n;
```

类模板应用举例

```
#include <iostream>
#include <cstdlib>
using namespace std;
class Student {
  public:
        int id;
        float gpa;
       Student(int i=0,float g=0):id(i), gpa(g) { }
};
template<typename T>  class Store
{       T item;
        int Valued;
 public:
      Store();
        T GetElem();
        void PutElem(T x);
};
```

定义类Student

类模板，实现对任意类型数据进行存取

类模板应用举例

构造函数

类模板成员函数定义

```
template<typename T>Store<T>::Store( ):Valued(0){ }
template<typename T>T Store<T>::GetElem()
{
    if(Valued==0)
   {
          cout<<"No item present!"<<endl;
          exit(1);
}
    return item;
}
template<typename T>void Store<T>::PutElem(T x)
{
    Valued++;
    item=x;
}
```

类模板外部定义的类模板的成员函数必须是函数模板

类模板应用举例

```
int main()
{  Student g(1000,23);                  //声明Student类型变量，赋以初值
   Store<int> S1,S2;                    //声明两个Store<int>类对象
   Store<Student> S3;                   //声明Store<student>类对象S3
   Store<double> D;                     //声明Store<double>类对象D
   S1.PutElem(3);
   S2.PutElem(-7);
    cout<<S1.GetElem()<<endl;
    cout<<S2.GetElem()<<endl;
    S3.PutElem(g);
   cout<<S3.GetElem().id<<endl;
   cout<<"Retrieving object D\n";
   cout<<D.GetElem()<<endl;             //输出对象D的数据成员
   return 0;    }
```

3、类模板

模板参数表中可以有非类型参数

模板类型参数

模板非类型参数

```
template<typename T, int size> class array
{
      T Element[size];
      int num;
    public:
       array():num(size){ };
};
```

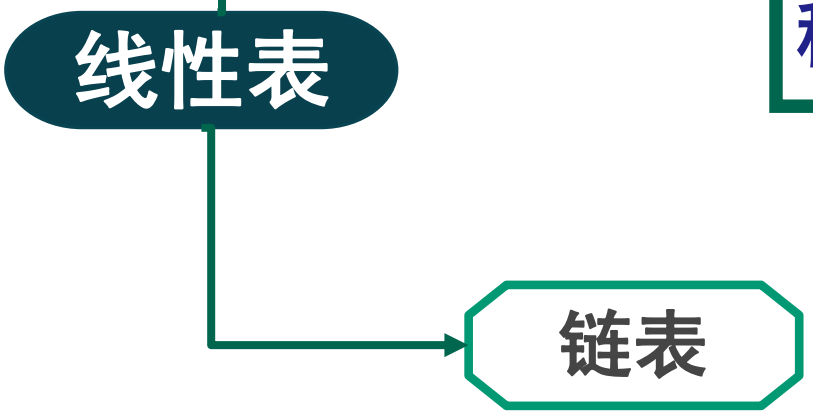
非类型参数通常代表了一个潜在的常量，不可修改该参数的值

```
array<int, 30>  a1;
array<double,40> a2;
```

4、线性表

线性关系

有限序列，相邻元素直接有直接前驱和直接后继的关系，且除了第一个和最后一个元素外，每个元素有且仅有一个直接前驱和直接后继。



一维数组元素处于一个线性表中，在内存中数组元素是顺序排列的，称为顺序表

静态数组：由编译器编译时分配内存，数组的长度不可改变。

防溢出：一般采用“大开小用”的方法，且应在添加新数据时判断表是否满

4.1、顺序表

顺序表的常见操作

- 计算表长度
- 判断某变量或对象x是否在表中
- 寻找某变量或对象x在表中的位置
- 插入
- 删除
- 寻找x的后继和前驱
- 判断表为空或者满
- 取第i个元素的值

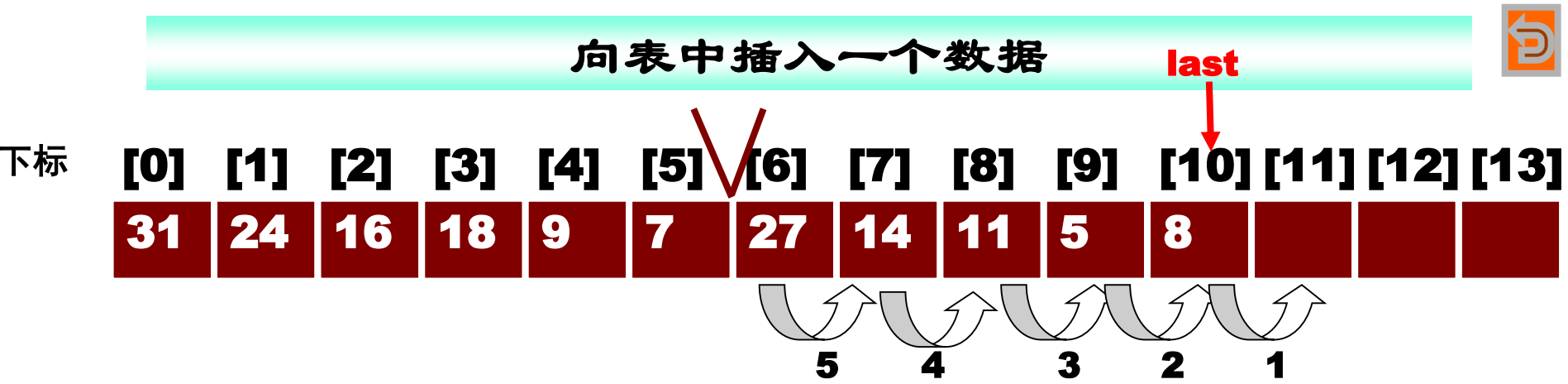
这些操作和数组可以封装在一起定义一个类

类模板

表内的元素可以是基本数据类型（int double。。。）变量，也可以是结构变量、类对象

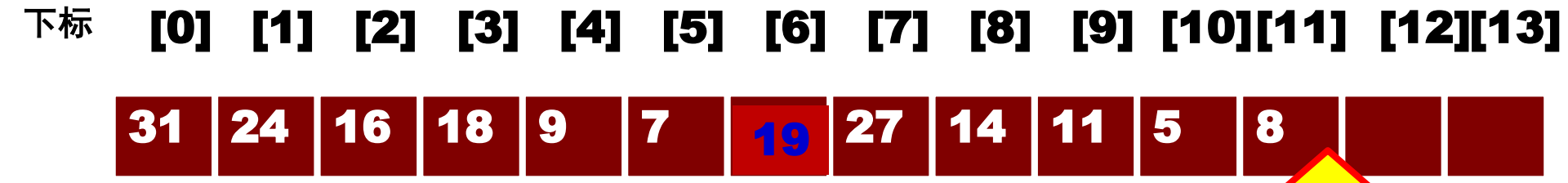
4.1、顺序表

向表中插入一个数据



X= 19

```
last++;
for(j=last;j>i;j--) a[j]=a[j-1];
```

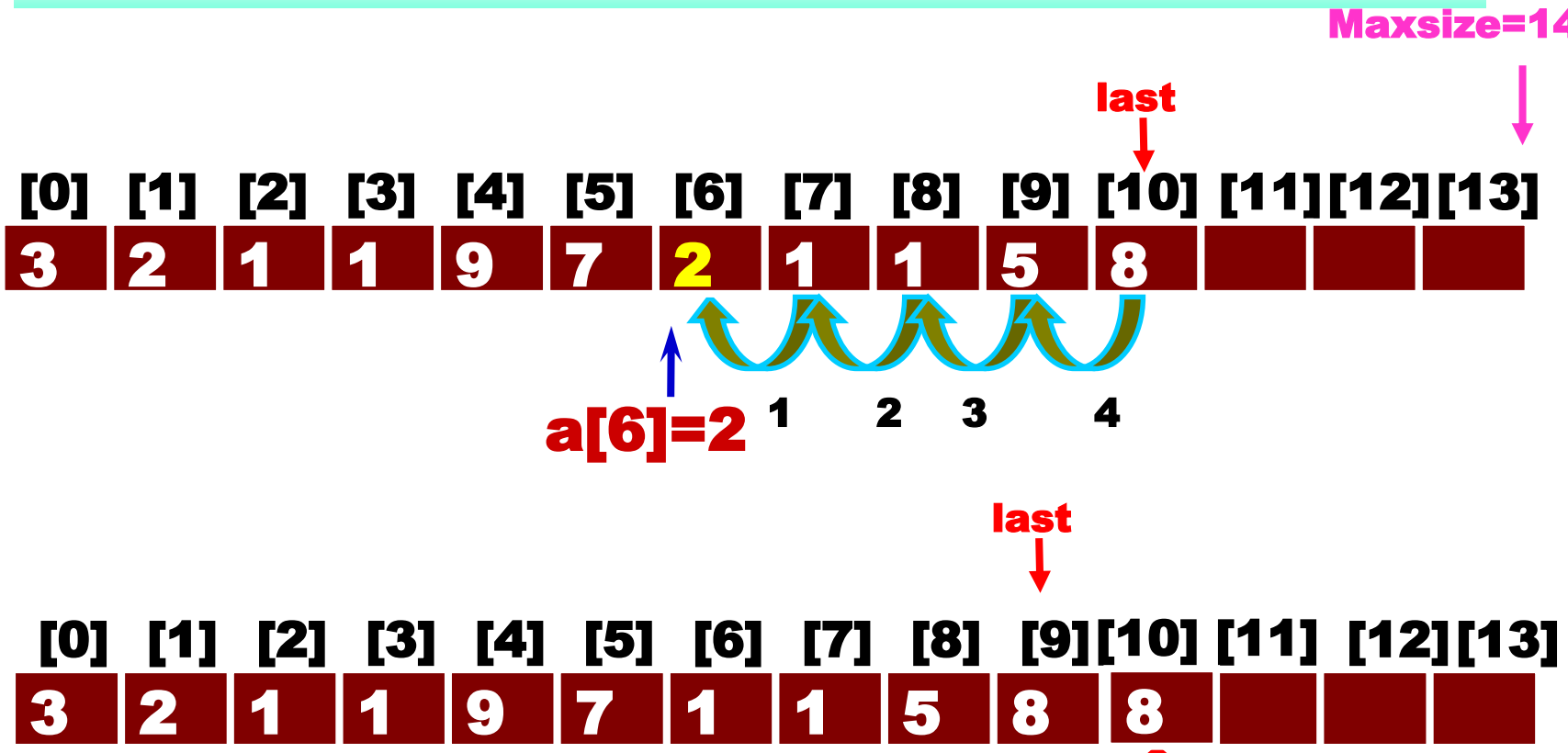



```
a[6]=19;
```

移动次序是关键

4.1、顺序表

从表中删除一个数据




```
last--;
for(j=6;j<=last;j++)
    a[j]=a[j+1];
```

移动次序是关键